MetaDiet AI

근거와 승인 경계를 가진 LangGraph 멀티에이전트 식단 코치

답변 생성에서
복구 가능한 서비스로

1인 개발 | AI Agent · RAG · Backend · Frontend · Infra · Evaluation

프로젝트 목표

음식량·영양 수치를 그럴듯하게 생성하거나 승인 없이 전략을 바꾸는 위험을 줄인다. 모델은 작업과 읽기 도구를 선택하고, 코드가 수치·권한·예산·근거·저장 승인을 통제한다. 일반 성인 체중 관리 정보 서비스이며 의료 진단·치료 서비스가 아니다.

현재 공개 결과

React/FastAPI 서비스를 Render 제한 베타로 배포했다. 식사·체중·인바디·목표 관리, HITL 확정, 대화 복원, 관리자 제품 분석과 LangSmith trace 연결을 구현했다.

대표 증거

독립 RAG 20문항: faithfulness 1.00, context recall 0.95.
독립 Agent 합성 8개 흐름: 목표 달성 8/8, 정책 위반 0.
동일 개발 8개 흐름: p95 24.92초 → 13.69초 (-45.1%).

공개 베타: metadiet-web.onrender.com
소스·증거: github.com/Kim-Du-Yeon/MetaDiet-AI
문서 기준 커밋: 2f992ed. 실제 사용자 만족도·건강 효과는 아직 미측정.

구현 기능과 책임 경계

모델 판단을 서비스의 데이터 변경 권한과 분리

사용자 흐름

가입·JWT 로그인 → 활동 수준/목표 설정 → 자연어 또는 사진 식사 입력 → 수치와 근거 확인 → 수정·확정 → 대시보드 조회. 사진 인식은 파싱에만 사용하며 중량과 저장은 사용자가 확인한다.

측정 데이터 분리

일상 체중은 weight_logs, 완전한 체성분은 inbody_logs에 보관한다. 가입 체중을 첫 기준점으로 표시하고 인바디 확정 시 체중을 연동한다. 골격근량·체지방률이 없으면 미입력으로 표시하고 임의로 추정하지 않는다.

LangGraph 실행 구조

load_state → Gemini Supervisor → 전문 Agent/검증 도구 → Validator → versioned HITL proposal. Meal Planner/Validator 피드백은 최대 3회로 제한한다. Weekly Deep Agent는 장기 기록과 읽기 전용 MCP를 별도 예산 안에서 분석한다.

코드의 안전 경계

CSV 영양 계산, JWT 소유권, 날짜·수치 검증, 실행 예산, 근거 충돌과 proposal ID/version을 검사한다. 승인 전에는 domain DB를 쓰지 않는다. 취소·중복 확정·재시도는 멱등 정책을 따른다.

화면과 복구

최근 대화·섭취량·진행 proposal을 체크포인트에서 복원하고 완료 실행은 재수행하지 않는다. 기존 설정 조회는 로딩/오류/신규 입력 상태를 나눠 온보딩 폼의 순간 노출을 제거했다.

RAG 구성 선정과 독립 평가

10개 PDF · 1,953페이지 · 4,139 활성 청크

비교하고 선택한 구성

PyPDF/PDFPlumber/Docling과 page/parent-child/section/recursive 청킹을 비교했다. Gemini embedding-2의 768/1536/3072 MRL prefix, Dense/BM25/Hybrid, 리랭커도 비교했다. 최종 운영 구성은 PyPDF + recursive + 3072차원 + Kiwi BM25/Dense RRF Hybrid + 리랭커 없음이다.

평가 계약

구성 선정 후 동결한 독립 20문항/10개 이중 언어 사실 그룹을 사용했다. 검색 후보 20개에서 context 4개, 6,000자 제한. 생성은 gemini-3.8-flash, Ragas 0.2.15 judge는 gemini-3.1-pro-preview이다.

결과

지정 source/page Recall@4: 85% (17/20).
대체 유효 근거를 원문 검토로 허용한 support: 95%.
Source-reviewed correctness: 95% (19/20).
Ragas faithfulness: 1.00 / context recall: 0.95, 유효 20/20.
검색+답변 p50/p95: 11.23/25.02초 (로컬 평가 환경).

실패의 해석

영어 아동 외삽 문장 질문 1건은 실제 근거를 찾지 못했고 답변을 보류했다. 두 문항은 지정 페이지 대신 같은 범위의 WHO 대체 근거를 찾았다. strict Recall과 재판정 support를 합치지 않는다.

평가 한계

원문 correctness 검토는 Codex의 비맹검 사후 검토이며 독립 임상 전문가 검증이 아니다. 소수 문항의 자동 지표를 서비스 전체 정확도나 사용자 건강 효과로 표현하지 않는다.

Agent 최적화와 행동 평가

개발 전후 비교와 독립 최종 평가를 별도로 유지

중복 호출 제거

성공한 동일 읽기 도구 결과를 실행 내에서 캐시하고 Supervisor의 중복 종합 호출을 제거했다. 실패 결과, 쓰기, 위임, proposal 변경은 캐시하지 않는다. 품질·근거 검증·HITL은 유지했다.

동일 개발 8개 흐름 전후, 각 1회

p50: 16.64초 → 9.31초 (-44.0%).
p95: 24.92초 → 13.69초 (-45.1%).
모델 호출 합계: 35 → 30 (-14.3%).
목표 달성은 양쪽 8/8, 정책 위반 0.
변경 전 도구 호출 합계가 없어 도구 절감률·실제 청구 비용 절감은 주장하지 않는다.

별도 독립 합성 8개 흐름

목표 달성 8/8, 정책 위반 0, 승인 전 write 0.
p50 16.71초 / p95 28.05초. 모델 28회 / 도구 24회 합계.
흐름별 ≤8 모델·≤12 도구·≤60초 계약 충족. LangSmith root 8개, error 0.

Deep Agent/MCP 실패 주입

부분 기록, API/MCP 장애, 실행 예산 초과, 근거 충돌, 읽기 전용 도구의 쓰기 시도 5개 시나리오를 검증했다. 정책 위반과 승인 전 저장이 없음을 확인했다. scripted-model 시험이며 실제 Gemini 답변 품질 시험과 구분한다.

회귀 검증

현재 구현 이후 백엔드 389 passed, 1 skipped (선택적 파서 환경), 프론트 58 passed, production build 성공을 기록했다. 과거 실험 snapshot을 최신 코드의 독립 재평가로 소급해 표현하지 않는다.

실패 원인과 해결 과정

로그 → 원인 분리 → 대응 선택 → 재검증

FlashRank OOMKilled

API, BM25 4,139청크, Kiwi와 ONNX/PyTorch 런타임·입력 버퍼가 한 Pod의 1GiB 메모리 예산을 넘었다. Kubernetes는 한도를 집행한 것이며 근본 원인은 자원 구성이다. 품질도 낮아 메모리 증설 대신 운영 리랭커를 제거했다.

Docling native 오류 - 미해결 경계

Python stack dump 경쟁과 PDF parser C++ 접근 오류를 분리했다. 페이지/프로세스 격리와 PDFium 우회를 시험했다. 10개 PDF 처리는 끝났지만 접근 오류/dump가 남았다. 문제 24페이지 재실행은 오류 0, 174블록 일치였으나 근본 해결 증거가 아니므로 운영은 PyPDF를 유지했다.

SSE 도중 Pod 삭제

진행 journal은 Pod 메모리, 지속 실행 상태는 Postgres checkpoint에 존재한다. 같은 run ID로 다른 Pod에서 재개했다. 실제 시험에서 proposal 1개, 승인 전 meal 0개, 중복 승인 후 meal 1개를 확인했다.

공개 Gemini API_KEY_INVALID

LangSmith에서 Supervisor 아래 Gemini 실패를 추적했다. Render GEMINI_API_KEY에 OpenAI 키가 잘못 입력된 원인이었다. 올바른 Gemini 키로 교체하고 실제 요청을 재검증했다. 비밀 값은 문서·로그에 기록하지 않았다.

인증과 배포 오류

가입 메일의 localhost redirect는 current origin과 Supabase 허용 URL로 수정했다. 만료 JWT 401은 공통 auth-expired 처리로 로그인에 복귀한다. Render 저장소 접근 경고는 GitHub credential 재연결 뒤 push 자동 배포로 재검증했다.

공개 배포와 지연 분해

상담 응답과 경량 health, 로컬과 공개 측정을 구분

무료 Render cold start

15분 이상 휴면 뒤 첫 /health: 53,022.0ms (1회).
기동 후 /health 20회: 중앙값 236.2ms, p95 949.9ms,
범위 203.9~1,272.4ms. 이것은 상담 시간이 아니다.

앱 초기화와 플랫폼 복구

로컬 새 프로세스 → health 준비 5,720.3ms (1회).
import main 3회 중앙값 5,132.3ms. Python/Agent/SDK import가 앱 초기화 대부분이다. 서로 다른 환경이라 정확히 빼서 플랫폼 시간을 확정할 수는 없지만, 휴면 복구가 주 지연 요인임을 분리했다.

운영 선택

제한 베타에서는 cold start를 안내한다. health만 먼저 여는 lazy initialization은 첫 상담 지연을 옮길 수 있다. 인위적 keep-alive를 추가하지 않았다. 상시 응답이 필요하면 휴면하지 않는 인스턴스 전환이 판단 지점이다.

제한 부하와 복구 시험

공개 health 50건, 인증 조회 30건, Agent SSE 2건: 오류 0.
Agent 2개 동시 첫 이벤트 4,317.4/4,450.3ms,
전체 wall time 32.48초. 대규모 부하나 SLA 검증은 아니다.
kind rolling restart health 400/400, 실패 0, p95 5.86ms.
로컬 health 수치를 공개 상담 성능으로 표현하지 않는다.

실제 브라우저 검증

공개 Chromium에서 스트리밍, proposal 새로고침 복원, API 500, SSE 중단, JWT 401, 피드백 503과 모바일 390×844 화면을 검증했다. 피드백 최초 201/재전송 200, DB 1행, 계정 삭제 후 0행을 확인했다.

관측과 남은 한계

사용자 행동 지표에서 실제 실패 실행까지 추적

제품 분석의 정확성

DAU는 최근 24시간, WAU는 최근 7일로 cohort 조회 기간과 분리했다. 가입은 실제 Auth 가입일, 온보딩은 전략 존재, 첫 식사는 실제 식사 기록으로 계산한다. 오류 표시와 실패 완료의 이중 집계를 제거했다. 완료율은 completed 수신 비율이며 답변 정확도가 아니다.

LangSmith 실제 연결

graph UUID → checkpoint/SSE trace_id → 제품 이벤트 → LangSmith root span을 연결했다. 공개 합성 계정에서 ID 일치와 stream_run_id를 조회했다. 실제 관리자 계정의 1/7/30/90일 조회와 새로고침 재조회를 확인했고 일반 사용자 API는 403으로 차단했다.

개인정보와 알림 경계

이벤트에는 원문·음식명·이메일·건강 수치를 담지 않는다. 내부 사용자 FK는 존재하므로 완전한 익명 데이터가 아니다. 운영 trace 입력/출력은 숨긴다. 이벤트 UUID 재전송은 1행으로 유지하고 계정 삭제 시 cascade한다. Render 이메일 실패 알림 설정은 확인했으나 실제 장애 메일 전달 시험까지 주장하지 않는다.

아직 확보하지 않은 결과

실제 사용자 수·만족도·장기간 건강 효과는 미측정이다. 브라우저 분석은 best-effort라 종료·네트워크 장애로 누락될 수 있다. 무료 cold start, 영어 근거 누락, 오래된 react-scripts 빌드 의존성은 남은 한계다.

다음 증거와 원본 자료

동의된 제한 베타 피드백을 수집하고 실패 trace를 익명화한 재현 테스트로 전환한다. 동일 조건의 개선 전후를 추가한다.
GitHub 원본: PORTFOLIO_CASE_STUDY.md, PORTFOLIO_RESULTS.md, EVALUATION.md, ARCHITECTURE.md.
독립 RAG: docs/rag_experiments/RAG_FINAL_EVALUATION_V1.md.
운영 안내: docs/ADMIN_ANALYTICS_GUIDE.md.